

Technical Assignment

Assignment Title: Weekly Report Generator & Team Dashboard

Objective

Build a full-stack web application that allows individual team members to submit structured weekly work reports, allows managers to review those reports and send them back for correction when needed, and gives managers a consolidated dashboard to view and analyze reports across the whole team.

This assignment evaluates your ability to design multi-user systems with role-based access, implement a review/approval workflow, build clean UI components, structure backend services, integrate AI-assisted features (optional), and manage data efficiently.

You are free to choose your preferred stack, but modern frameworks such as React, Next.js, or similar frontend frameworks are preferred.

Core Requirements

1. User Authentication & Roles

Implement authentication with role-based access.

Required roles:

- **Team Member** - can create, edit, and submit their own weekly reports
- **Manager / Admin** - can view and analyze reports across all team members, and review/approve submitted reports

Required features:

- User registration
- Login / Logout
- Password protection
- Secure session handling
- Role assignment (e.g. by an admin, or at signup)

2. Personal Weekly Report Page

Every user must have their own dedicated page for creating and managing their weekly reports.

The report structure must be fixed and identical for every user - the same set of fields, in the same order, across the whole team. Users should not be able to customize, reorder, or add their own fields. This ensures reports stay consistent and comparable across the team on the manager's dashboard.

Each report should contain:

- Week / date range
- Project or category tag
- Tasks completed - with a task-level table (task name, priority, planned % vs actual %, status, time planned vs. time spent, output/deliverable produced)
- Tasks planned for next week
- Blockers / challenges (with the ability to flag one as the key issue for the week)
- Achievements / highlights (with the ability to flag one as the key achievement for the week)

- Hours worked, broken down by task type (e.g. Development, Testing, Meetings, Documentation) - optional
- Optional notes or links

Features required:

- Create weekly report (saved as a draft)
- Edit report while it is a draft or while it needs correction (see Section 3)
- Submit report for manager review
- View own report history, organized by week, with the current status of each (Draft / Submitted / Needs Correction / Approved)

3. Report Review & Correction Workflow

A report is not just submitted once - it goes through a review cycle between the team member and their manager. This workflow is a core, required part of the assignment, not optional.

Required status flow:

- **Draft** - team member is still filling in the report; only visible to them
- **Submitted** - team member has submitted the report for review; it now appears on the manager's dashboard
- **Needs Correction** - the manager reviewed the report and sent it back with a general comment describing what needs to change; the report becomes editable again by the team member
- **Approved** - the manager is satisfied with the report; no further edits are expected

Required behavior:

- A manager reviewing a submitted report can either Approve it, or Request Changes by leaving one general comment explaining what needs correction.
- When a manager requests changes, the report status changes to Needs Correction and the team member must be able to see the manager's comment clearly on their report page.
- The team member can then edit the same report and resubmit it, which moves the status back to Submitted for another review.
- Team members must only be able to see and edit their own reports. Managers must be able to see every team member's reports, but should only be able to edit the status/comment fields - not rewrite the team member's actual report content.

Bonus (not required): keep a short history of previous review comments per report, rather than only the latest one.

Report version history: each time a report goes through a correction cycle (Needs Correction → edited → resubmitted), the previous version of that report's content must remain visible, not just overwritten. A manager reviewing a report must be able to clearly see each past version of that week's report alongside the version currently under review, and which version a given comment was made against. This does not need to be a full diff/comparison view - a simple list of past versions with submission timestamps, viewable on demand, is sufficient.

4. Team Dashboard (Manager View)

Managers should be able to view and analyze reports submitted by the whole team.

Features:

- View all team members' reports for a selected week
- Filter reports by team member
- Filter reports by project / category
- Filter reports by date range
- Filter/track submission status per team member (Draft / Submitted / Needs Correction / Approved / not yet started)
- Open any report to review its full contents and take a review action (Approve / Request Changes)

Good to have (bonus): for a selected week, let the manager view one section (e.g. Blockers, or Achievements) across all team members side by side, rather than opening each report individually.

5. Projects / Categories

Allow projects or work categories to be managed and attached to report entries.

Examples: Client A, Internal Tooling, R&D, Marketing.

Features:

- Add project / category
- Edit project / category
- Delete project / category
- Assign team members to relevant projects (optional)

6. Dashboard & Visual Insights

A data-driven dashboard must be implemented for managers.

Summary metrics:

- Total reports submitted this week
- Submission compliance rate (submitted vs pending vs late)
- Number of reports currently in Needs Correction status
- Number of open blockers across the team

Visual insights - include charts such as:

- Tasks completed trend over time (per person or team-wide)
- Report submission/approval status by team member
- Workload / task distribution by project
- Time spent by task type, team-wide (e.g. how much time went to meetings vs. development)
- Recent reports / activity feed, including recent review actions (approved / sent back for correction)

Candidates may use any charting library. Examples: Recharts, Chart.js, E-Charts.

7. Additional Pages Required

Beyond the core report and dashboard pages described above, implement at least 7 of the following pages/views:

- Login / Register page
- Personal weekly report page (create / edit)
- Report history page (per user) - a list view of past reports and their statuses, separate from the create/edit page
- Report detail / view page - a read-only view of a single report, used by both team members and managers
- Team member profile page (manager view) - clicking a team member shows their full report history and basic stats
- Project / category management page - a proper page with a list and CRUD actions, not just a modal
- User management page (admin) - invite/remove team members, assign roles
- Manager review page - where a manager opens a submitted report and takes an Approve / Request Changes action with a comment

Feel free to add any other pages that make sense for the product (e.g. account settings) - use your own judgment here.

Submissions with only two or three basic screens (e.g. just a form and a dashboard) will be considered incomplete, even if the core logic works.

8. AI Chat Assistant [Good to Have]

As an optional enhancement, candidates may integrate an AI-powered chat assistant into the application. This is not mandatory but will be viewed favorably during evaluation.

Suggested capabilities:

- Conversational Q&A for managers about team activity (e.g. "What did the design team work on last week?")
- AI-generated team summary highlighting completed work, recurring blockers, and workload imbalances
- Simple in-app chat widget UI for interacting with the assistant

Candidates may use any LLM provider (e.g. Anthropic Claude, OpenAI, or an open-source model) and any integration approach (direct API calls, function calling / tool use, or a lightweight RAG setup over stored reports). Please document your approach, prompt design, and any data-privacy considerations in the presentation if this feature is implemented.

Technical Requirements

Frontend

Preferred frameworks:

- React
- Next.js
- Angular
- Vue

UI expectations:

- Responsive layout
- Component-based structure
- Clean, reusable components
- Well-structured folder organization
- Clear separation between the personal report page, report history, and the team dashboard
- Basic client-side form validation

Backend

Preferred backend frameworks:

- Spring Boot
- C# (.NET / ASP.NET Core)
- Node.js (Express or NestJS)
- Python (FastAPI or Django)

Required:

- REST API
- Proper request validation
- Role-based access control on relevant endpoints (a team member must never be able to access another team member's report data or a manager-only endpoint)
- Clean service / controller structure
- Pagination and/or filtering on any endpoint returning a list of reports

Database

You may choose any database. Examples:

- PostgreSQL
- MySQL
- MongoDB

Your schema should clearly represent users, roles, projects, reports, and the review/status history of each report (at minimum, current status and the latest reviewer comment).

Scope & Difficulty Expectations

This assignment is intended to reflect a realistic small internal tool, not a quick prototype. To be evaluated fairly, your submission should demonstrate:

- At least 7 of the pages/views listed in Section 7, implemented and connected to real backend data
- A working end-to-end review cycle: submit → manager requests changes → team member edits and resubmits → manager approves
- A seeded dataset with at least 3–5 team members and several weeks of reports in different statuses, so the dashboard is meaningful to review
- At least one automated test covering your role-based access control logic (bonus, but strongly recommended)
- A deployed, publicly accessible instance of the application (bonus - local-only submissions are accepted, but a deployed link is viewed favorably)

Deliverables

Candidates must submit the following:

1. Presentation

Candidates must prepare a short technical presentation explaining their solution. The presentation should cover:

- System architecture overview
- Database design explanation
- Key frontend components (personal page vs. report history vs. team dashboard)
- API design and backend structure, including role-based access
- How the report review/correction workflow was implemented
- AI Chat Assistant approach, if implemented (Good to Have)
- Challenges faced and how they were solved
- Possible future improvements

Accepted format: Google Slides, shared via the Google Drive folder (see Submission section) with sharing access enabled.

2. GitHub Repository

The repository must contain:

- Frontend code
- Backend code
- Setup Instructions README file covering: 1) Installing dependencies, 2) Running frontend, 3) Running backend, 4) Running database

3. ER Diagram

Provide an Entity Relationship Diagram that explains your database design, including how users, roles, projects, reports, and report status/review history relate to one another.

Accepted format: an image file of the diagram, uploaded to the shared Google Drive folder (see Submission section) with sharing access enabled.

4. Video Explanation / Demo

If the application is fully or partially developed, candidates must submit a short video explanation (demo) of the application. Please ensure your camera is turned on so that your face is visible while you present your project. The video should ideally cover:

- Application walkthrough (as both a Team Member and a Manager, if roles are implemented)
- The full review/correction cycle for at least one report
- Report details for 2-3 different team members, to show the data is genuinely multi-user
- Key features implemented
- Frontend flow
- AI Chat Assistant demo, if implemented (Good to Have)

There is no need to show raw database records or run database queries on camera - the walkthrough should stay in the application UI.

Accepted format: video file, uploaded to the same shared Google Drive folder as the presentation and ER diagram, with sharing access enabled.

Evaluation Criteria

Your solution will be evaluated based on:

- System design
- Code quality
- UI structure and completeness (see Section 7 page requirements)
- Component reusability
- API design, including role-based access control
- Database design
- Correctness and completeness of the review/correction workflow
- Documentation clarity
- Technical explanation during the presentation
- Genuine understanding of your own codebase, as demonstrated in the live coding round
- *AI Chat Assistant implementation (bonus points, not mandatory)*

Submission

Create a single Google Drive folder containing your presentation (Google Slides), ER diagram (image), and demo video. Enable sharing access ("Anyone with the link") on the folder before submitting.

In the body of your submission email, please include:

- Your GitHub repository link
- The shared Google Drive folder link (presentation, ER diagram, and video)

Note: submissions that are incomplete, sent through the wrong channel, missing required links, or with links that are not accessible (sharing permissions not enabled) will not be evaluated, and this on its own is sufficient grounds for not being selected to proceed. Please double-check your links and formats before submitting.

Note on AI Tool Usage & Live Coding Round

You are welcome to use AI tools while building this assignment - this is a normal part of modern development and is not penalized.

However, shortlisted candidates will be invited to a live coding round, where you will be asked to explain your codebase in detail and make real-time changes or add a small new feature to the application you submitted. This round is designed to verify genuine understanding of your own solution, regardless of how it was built. Please make sure you can confidently explain and extend every part of what you submit.